



UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA

Alexandre Pedroza (202400304)
João Guilherme Faria (202406919)
Lucca Magnino (202401805)
Sophia Luna (202406926)

ATIVIDADE 2: Comunicação de IoT utilizando MQTT

Goiânia
27/07/2026

1. INTRODUÇÃO

A Atividade 1 propôs uma rede de estações submersas para detecção precoce do peixe-leão (*Pterois volitans*) no Grande Sistema de Recifes da Amazônia (GSRA), espécie invasora sem predadores naturais no Atlântico Sul e cujo controle hoje depende quase exclusivamente da remoção física reativa. A arquitetura definida organiza-se em camadas: cada estação submersa concentra sensores ambientais (temperatura, salinidade, turbidez, profundidade) e uma câmera subaquática processados na borda por um ESP32 (aquisição) e um NVIDIA Jetson Nano (visão computacional com modelo YOLO); os dados sobem por cabo umbilical até uma boia de superfície que atua como gateway, transmitindo por satélite IoT até uma plataforma em nuvem baseada em AWS IoT Core e AWS Timestream, disponibilizada em dashboard web e aplicativo móvel para ICMBio, IBAMA, pesquisadores e pescadores.

Nesta Atividade 2, o objetivo é detalhar a camada de comunicação dessa arquitetura, definindo explicitamente os papéis de Publisher, Broker e Subscriber no protocolo MQTT, especificando os tópicos e o fluxo de mensagens, realizando uma prova de conceito com um broker público e planejando a organização do projeto em um repositório GitHub, além da integração futura com o AWS IoT Core.

2. ARQUITETURA ATUALIZADA

A arquitetura da Atividade 1 é mantida, mas agora detalhada sob a ótica do protocolo MQTT (Message Queuing Telemetry Transport), adequado ao cenário por ser leve, orientado a publicação/assinatura e tolerante a conexões intermitentes, características centrais para o backhaul via satélite IoT da estação submersa.

Quanto ao primeiro componente novo, a própria estação submersa (ESP32 + Jetson Nano, via boia-gateway) atua como Publisher, publicando leituras ambientais, eventos de detecção e mensagens de status/heartbeat.

Para a prova de conceito desta atividade, utiliza-se um broker MQTT público (HiveMQ, broker.hivemq.com, porta 1883, sem autenticação). Em produção, esse papel será exercido pelo AWS IoT Core, com autenticação mútua via certificados X.509.

Do lado da nuvem, atuam como Subscribers: o motor de regras do AWS IoT Core (que grava as leituras no AWS Timestream e aciona alertas), o dashboard web (mapa de detecções) e o aplicativo móvel (notificações push para as equipes de campo).

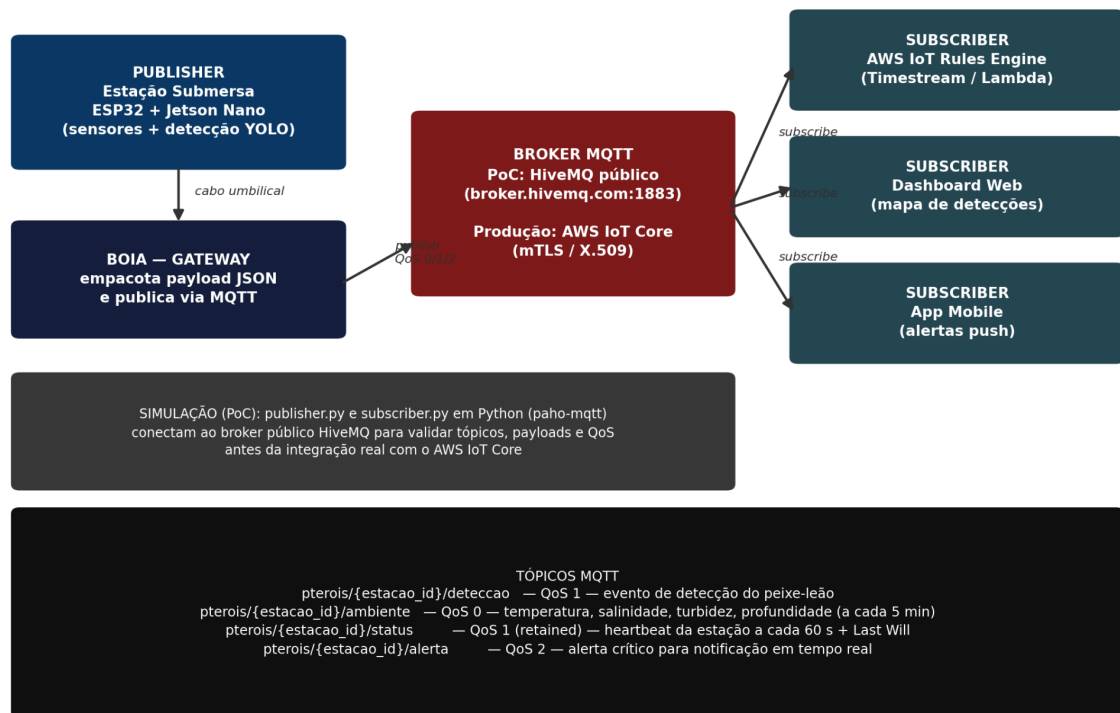


Figura 1 — Arquitetura de comunicação MQTT proposta (Publisher / Broker / Subscriber)

3. DEFINIÇÃO DOS TÓPICOS MQTT

Os tópicos foram organizados em uma árvore hierárquica prefixada por *pterois/{estacao_id}/...*, permitindo que múltiplas estações publiquem na mesma rede e que os subscribers utilizem wildcards (ex.: *pterois/+/{deteccao}*) para consumir dados de toda a rede de sensores. As mensagens são transmitidas em formato JSON. O nível de QoS foi escolhido de acordo com a criticidade e a frequência de cada tipo de mensagem: QoS 0 para leituras ambientais periódicas e não críticas (perda tolerável), QoS 1 para eventos de detecção e status (entrega garantida, duplicação tolerável) e QoS 2 para alertas críticos, em que nem a perda nem a duplicação da mensagem são desejáveis.

Tópico	QoS	Frequência	Descrição
pterois/{estacao_id}/deteccao	1	Evento (assíncrono)	Evento de detecção do peixe-leão pelo modelo YOLO (Jetson Nano): timestamp, coordenadas, nº de indivíduos, confiança.
pterois/{estacao_id}/ambiente	0	A cada 5 min	Leituras ambientais consolidadas pelo ESP32: temperatura, salinidade,

			turbidez e profundidade.
pterois/{estacao_id}/status	1	A cada 60 s	Heartbeat da estação, com nível de bateria; usa Last Will and Testament (LWT) para sinalizar queda de conexão.
pterois/{estacao_id}/alerta	2	Evento crítico	Alerta de alta prioridade disparado quando a confiança da detecção ultrapassa o limiar definido (0,75); consumido pelo dashboard e pelo app para notificação push.

Tabela 1 — Tópicos MQTT, QoS e frequência de envio

O tópico de status utiliza a flag retained (para que um novo subscriber saiba imediatamente o último estado conhecido da estação) e é associado a uma mensagem de Last Will and Testament (LWT), configurada na conexão do Publisher: caso a estação perca a conexão de forma inesperada, o próprio broker publica automaticamente uma mensagem de status "offline" em nome dela, permitindo detecção de falhas de equipamento.

4. FLUXO DE COMUNICAÇÃO

O ESP32 realiza a leitura periódica dos sensores ambientais e o Jetson Nano executa a inferência do modelo YOLO sobre o vídeo da câmera subaquática. A cada 5 minutos, a estação publica no tópico ambiente, a cada 60 segundos, publica um heartbeat no tópico status (retained). Quando o modelo detecta o peixe-leão, a estação publica imediatamente no tópico *deteccao* (QoS 1); se a confiança da detecção for igual ou superior a 0,75, publica adicionalmente no tópico alerta (QoS 2). O broker MQTT (HiveMQ na PoC; AWS IoT Core em produção) recebe as publicações e as encaminha a todos os Subscribers inscritos nos respectivos tópicos.

No lado da nuvem, as regras do AWS IoT Core persistem as leituras e detecções no AWS Timestream e acionam uma função Lambda para o tópico alerta, que dispara notificação push via Amazon SNS. O dashboard web e o aplicativo móvel, inscritos nos tópicos relevantes, atualizam o mapa de detecções em tempo real e notificam as equipes cadastradas. Após a remoção do indivíduo em campo, a equipe registra a ação no aplicativo, fechando o ciclo iniciado na detecção.

5. SIMULAÇÃO

Para validar o desenho de tópicos, o formato das mensagens e os níveis de QoS antes de qualquer custo ou configuração na AWS, foi implementada uma simulação em Python com a biblioteca `paho-mqtt`, conectando-se ao broker público HiveMQ (`broker.hivemq.com:1883`). Dois scripts compõem a simulação: `publisher.py`, que representa a estação submersa e publica leituras ambientais simuladas, heartbeats e eventos de detecção/alerta; e `subscriber.py`, que representa os consumidores em nuvem (equivalentes ao AWS IoT Core e ao dashboard), inscrevendo-se em `pterois/+/#` e exibindo as mensagens recebidas, classificadas por tipo.

Código 1 — `publisher.py` (publicação de detecção e alerta)

```
def publicar_deteccao_e_alerta(client, deteccao): payload = {
    "estacao_id": ESTACAO_ID,
    "timestamp": now_iso(),
    "lat": LAT,
    "lon": LON,
    "individuos": deteccao["individuos"],
    "confianca": deteccao["confianca"],
}
client.publish(TOPIC_DETECCAO, json.dumps(payload), qos = 1)
if deteccao["confianca"] >= LIMIAR_CONFIANCA: alerta = {
    "estacao_id": ESTACAO_ID,
    "timestamp": now_iso(),
    "mensagem": "Peixe-leão detectado com alta confiança",
    "lat": LAT,
    "lon": LON,
    "confianca": deteccao["confianca"]
}
client.publish(TOPIC_ALERTA, json.dumps(alerta), qos = 2)
```

Código 2 — `subscriber.py` (classificação das mensagens recebidas)

```
def on_message(client, userdata, msg):
```

```
def on_message(client, userdata, msg): payload = json.loads(msg.payload.decode())
tipo = msg.topic.split("/")[2]
if tipo == "alerta": print(f "ALERTA PUSH: {payload}")
elif tipo == "deteccao": print(f "DETECÇÃO: {payload}")
elif tipo == "ambiente": print(f "AMBIENTE: {payload}")
elif tipo == "status": print(f "STATUS: {payload}")
```

O código completo dos dois scripts, junto ao arquivo requirements.txt, está disponível no repositório GitHub do projeto (Etapa 5) e também anexado a esta entrega.

6. DISPONIBILIZAÇÃO

O projeto foi estruturado para um repositório GitHub denominado pterois-iot, contendo a documentação das Atividades 1 e 2, o diagrama de arquitetura, o código-fonte da simulação e um README com instruções de execução, conforme a estrutura abaixo.

```
pterois-iot/
├── README.md
├── docs/
│   ├── atividade1-planejamento.pdf
│   ├── atividade2-mqtt.pdf
│   └── diagrama-arquitetura-mqtt.png
├── src/
│   ├── publisher.py
│   ├── subscriber.py
│   └── requirements.txt
└── LICENSE
```

O README documenta o objetivo do projeto, a tabela de tópicos MQTT (Tabela 1), instruções para instalar as dependências (pip install -r src/requirements.txt) e executar a simulação em dois terminais (subscriber.py e, em seguida, publisher.py), além dos próximos passos de integração com a AWS descritos na Etapa 6.

7. PLANEJAMENTO DE INTEGRAÇÃO

A migração do broker público para a AWS IoT Core está planejada nas seguintes etapas: Provisionamento de uma "coisa" (Thing) por estação no AWS IoT Core, com certificado X.509 exclusivo e política IAM restringindo a publicação/assinatura apenas aos tópicos *pterois/{estacao_id}/**. Substituição do endereço do broker público pelo endpoint MQTT do IoT Core (mTLS na porta 8883) nos *scripts publisher.py*. Criação de IoT Rules que capturam mensagens de detecção e ambiente via consulta SQL sobre os tópicos e as inserem no AWS Timestream para análise histórica. Criação de uma regra adicional sobre o tópico alerta que aciona uma função AWS Lambda, responsável por publicar notificações via Amazon SNS para o aplicativo móvel e por atualizar o dashboard em tempo real. Uso do recurso de Device Shadow do IoT Core para manter o último estado conhecido de cada estação (equivalente ao tópico status retained) mesmo quando ela está offline. Configuração do IoT Device Defender para monitorar continuamente o comportamento de conexão das estações e identificar anomalias de segurança.

Essa migração preserva o desenho de tópicos, payloads e níveis de QoS validados na simulação com o HiveMQ, alterando apenas a camada de transporte e autenticação, o que confirma a utilidade da prova de conceito como etapa intermediária antes do custo e da complexidade operacional da nuvem.

8. CONCLUSÃO

A evolução da Atividade 1 para a Atividade 2 tornou explícita a camada de comunicação da solução de monitoramento do peixe-leão, definindo os papéis de Publisher, Broker e Subscriber, um esquema de tópicos hierárquico com níveis de QoS adequados à criticidade de cada mensagem, e uma prova de conceito funcional com um broker público. O planejamento de integração com o AWS IoT Core demonstra que a arquitetura proposta é compatível com um ambiente de produção seguro e escalável, mantendo o foco original do projeto: viabilizar a detecção precoce e a resposta rápida à invasão biológica do peixe-leão no Grande Sistema de Recifes da Amazônia.

REFERÊNCIAS

1- Stam, Gilberto. Peixe-leão, predador venenoso e de origem asiática, invade litoral brasileiro. Revista Pesquisa. 11 de setembro de 2023. Acesso realizado pelo [Site Oficial](#).

- 2- Silva, Joanna; Silva, Thayse Karla; Nascimento, Yuri Kayky. Impactos Ambientais do Peixe-Leão *Pterois volitans* (Linnaeus, 1758) em Ambientes Costeiros: Um alerta para o litoral nordestino do Brasil. Recife, 2023.
- 3- Atlam, Hany; Walters, Robert; Wills, Gary. Internet of Things: State of the art, Challenges, Applications, and Open Issues. International Journal of Intelligent Computing Research. Setembro 2018.
- 4- Draw Io. Diagrams Net. Acesso realizado pelo [Site Oficial](#).